Chapter 12
Microkernel Architecture Style

Microkernel Architecture Style

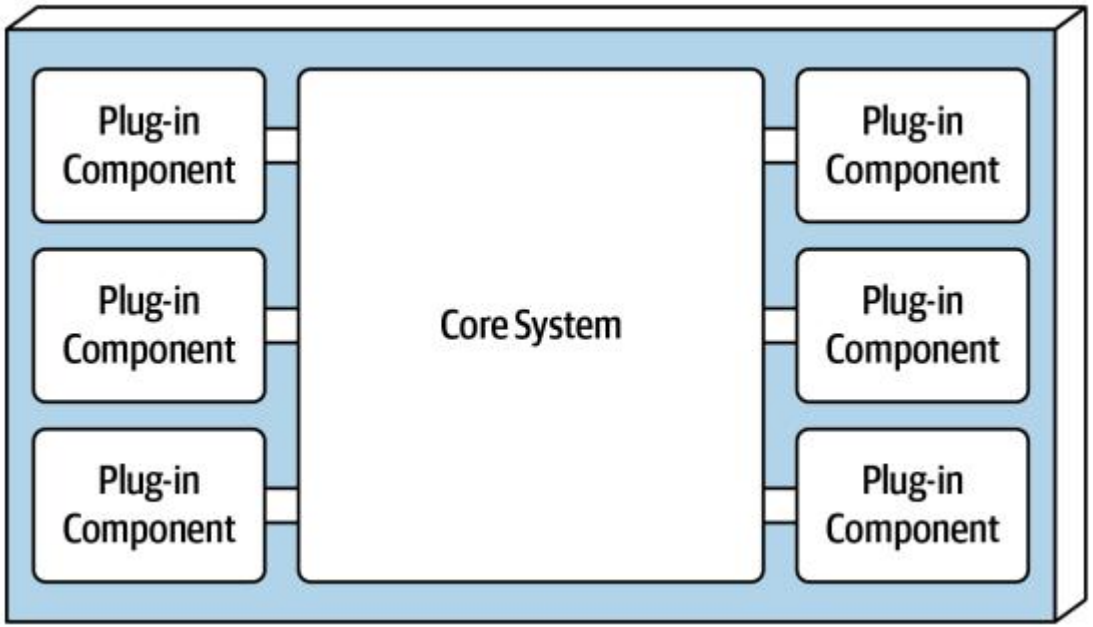
Introduction

- The microkernel architecture style (also referred to as the plug-in architecture) was coined several decades ago and is still widely used today.
- This architecture style is a natural fit for product-based applications (packaged and made available for download and installation as a single, monolithic deployment, typically installed on the customer's site as a third-party product) but is widely used in many nonproduct custom business applications as well.

Microkernel Architecture Style

Topology

- The microkernel architecture style is a relatively simple monolithic architecture consisting of two architecture components: a core system and plug-in components.
- Application logic is divided between independent plug-in components and the basic core system, providing extensibility, adaptability, and isolation of application features and custom processing logic.



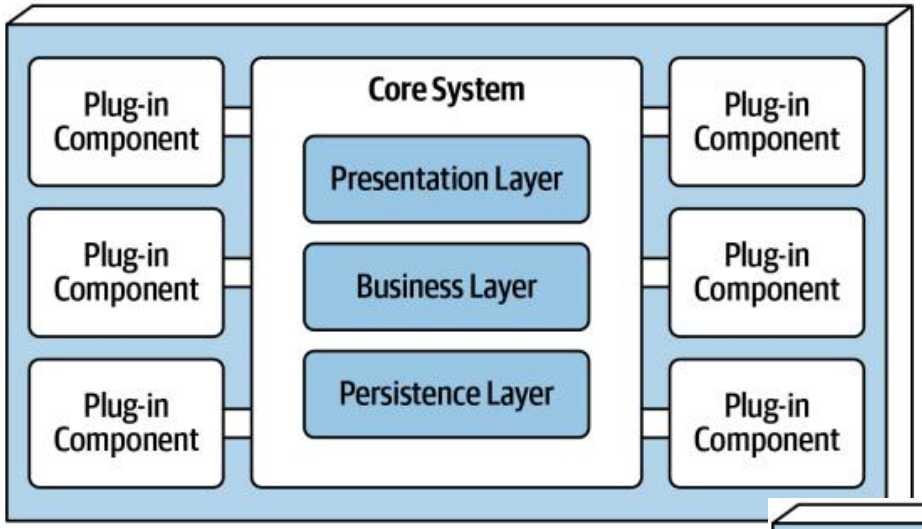
Microkernel Architecture Style

Core System

- The core system is formally defined as the minimal functionality required to run the system.
- Example: Eclipse IDE
  - The core system of Eclipse is just a basic text editor: open a file, change some text, and save the file. It's not until you add plug-ins that Eclipse starts becoming a usable product.
- Core system may be a happy path (general processing flow) through the application, with little or no custom processing.
  - Removing the cyclomatic complexity of the core system and placing it into separate plug-in components allows for better extensibility and maintainability, as well as increased testability.

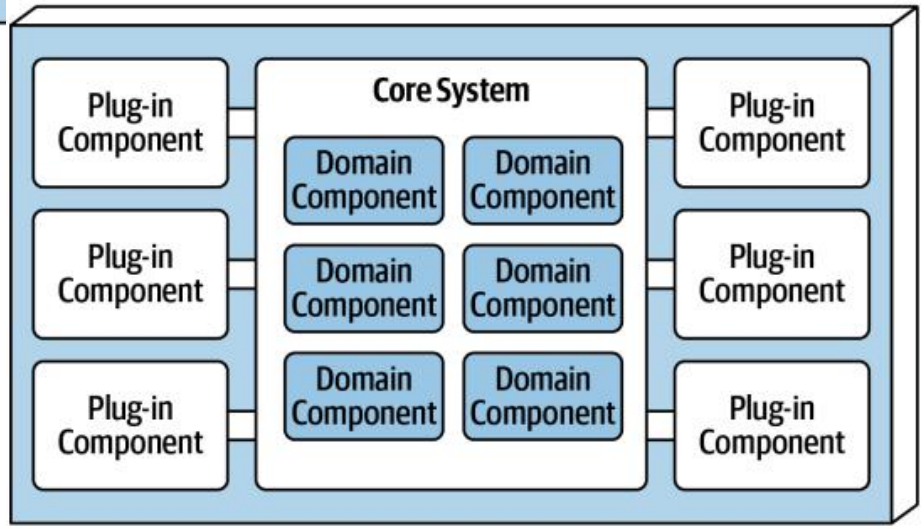
Microkernel Architecture Style

Core - Variations



Layered Core System (Technically Portioned)

Figure 12-2. Variations of the microkernel architecture core system

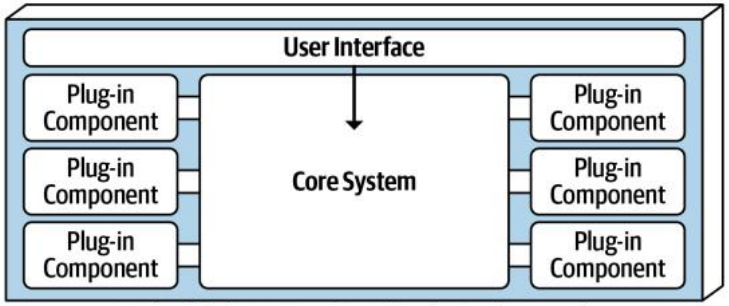


Modular Core System (Domain Portioned)

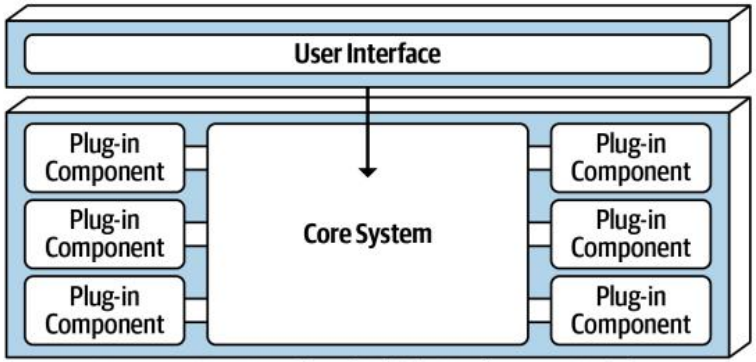
Microkernel Architecture Style

Core System - Presentation layer

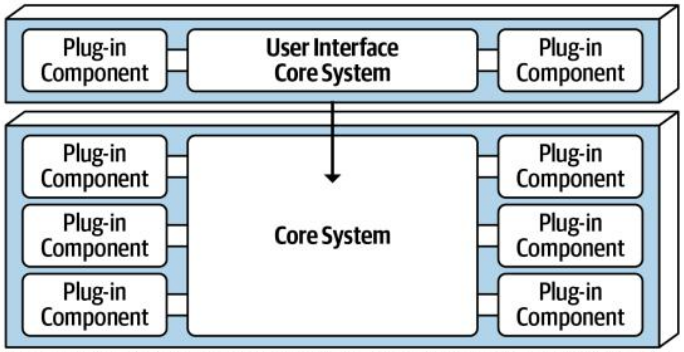
- The presentation layer of the core system can be embedded within the core system or implemented as a separate user interface, with the core system providing backend services.
- As a matter of fact, a separate user interface can also be implemented as a microkernel architecture style. Figure 12-3 illustrates these presentation layer variants in relation to the core system



Embedded User Interface (Single Deployment)



Separate User Interface (Multiple Deployment Units)



Separate User Interface (Multiple Deployment Units, Both Microkernel)

Microkernel Architecture Style

Plug-In Components

- Plug-in components are standalone, independent components that contain specialized processing, additional features, and custom code meant to enhance or extend the core system.
- Additionally, they can be used to isolate highly volatile code, creating better maintainability and testability within the application. Ideally, plug-in components should be independent of each other and have no dependencies between them.
- The communication between the plug-in components and the core system is generally point-to-point, meaning the "pipe" that connects the plug-in to the core system is usually a method invocation or function call to the entry-point class of the plug-in component.
- In addition, the plug-in component can be either compile-based or runtime-based.
  - Runtime plug-in components can be added or removed at runtime without having to redeploy the core system or other plug-ins, and they are usually managed through frameworks such as Open Service Gateway Initiative (OSGi) for Java, Penrose (Java), Jigsaw (Java), or Prism (.NET).
  - Compile-based plug-in components are much simpler to manage but require the entire monolithic application to be redeployed when modified, added, or removed

Microkernel Architecture Style

Plug-In Components – P2P

- Point-to-point plug-in components can be implemented as shared libraries (such as a JAR, DLL, or Gem), package names in Java, or namespaces in C#. Continuing with the electronics recycling assessment application example, each electronic device plugin can be written and implemented as a JAR, DLL, or Ruby Gem (or any other shared library)

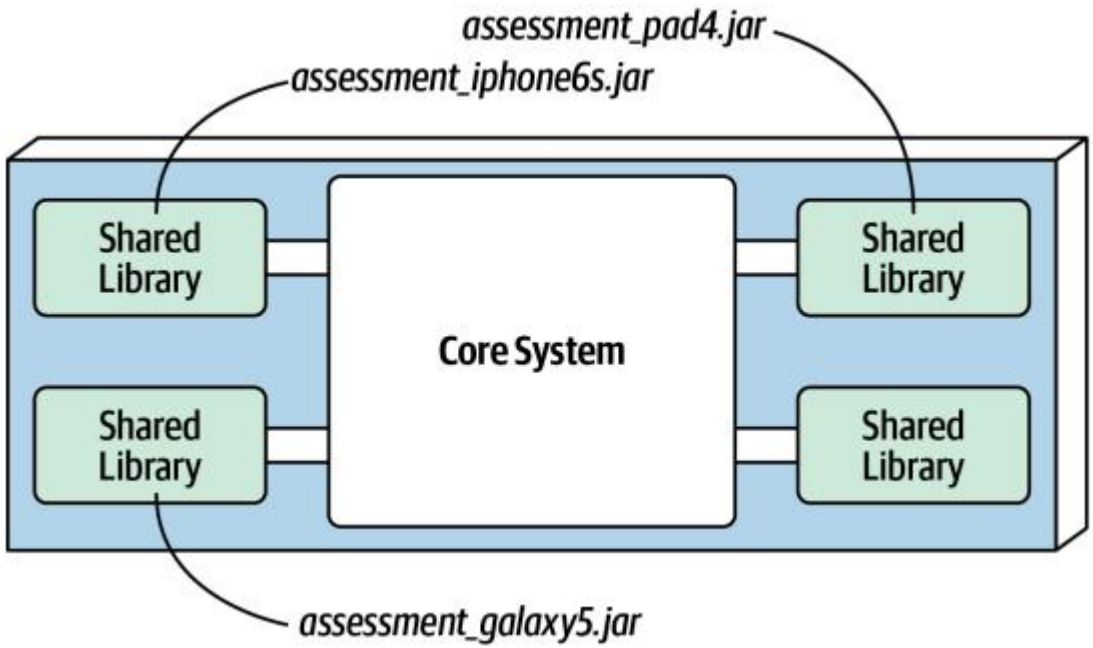


Figure 12-5. Package or namespace plug-in implementation

Microkernel Architecture Style

Plug-In Components – remote

- This type of plugin may including using REST or messaging to invoke plug-in functionality, with each plug-in being a standalone service (or maybe even a microservice implemented using a container).
- Although this may sound like a good way to increase overall scalability, note that this topology is still only a single architecture quantum due to the monolithic core system. Every request must first go through the core system to get to the plug-in service.

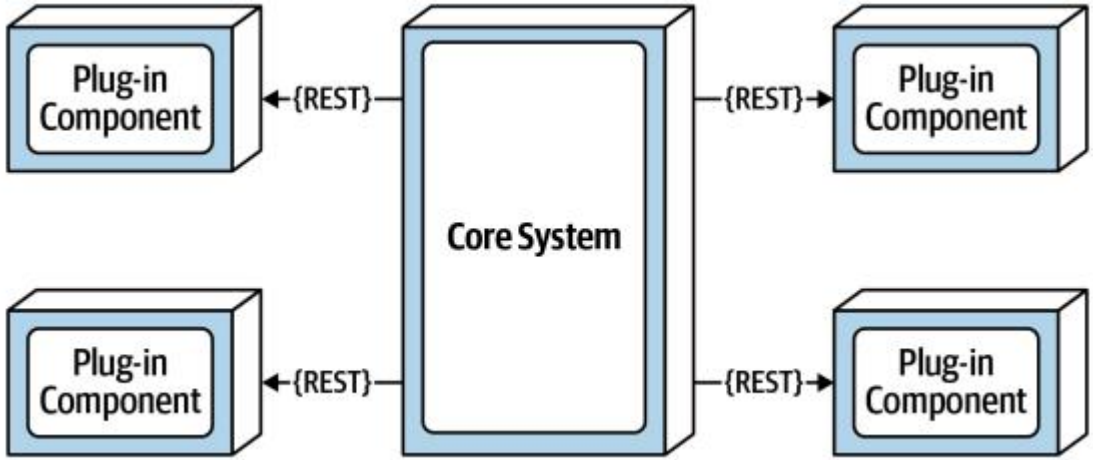


Figure 12-6. Remote plug-in access using REST

Microkernel Architecture Style

Plug-In Components

- With these benefits comes trade-offs. Remote plug-in access turns the microkernel architecture into a distributed architecture rather than a monolithic one, making it difficult to implement and deploy for most third-party on-prem products.

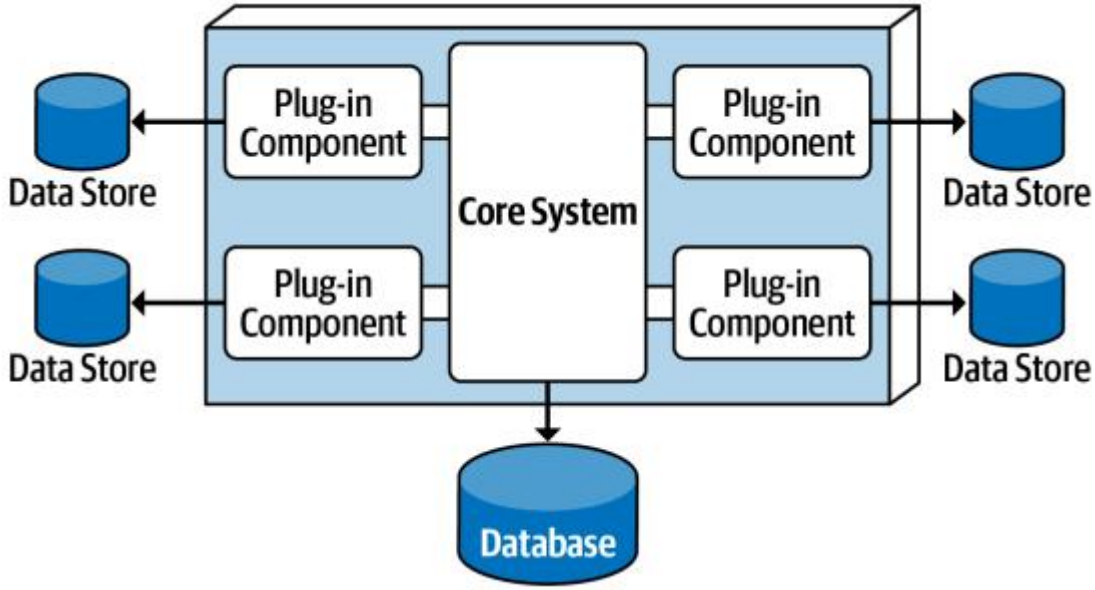


Figure 12-7. Plug-in components can own their own data store

Microkernel Architecture Style

Registry

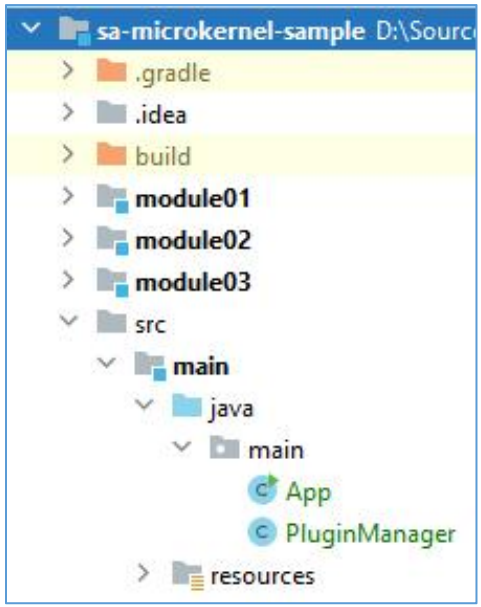
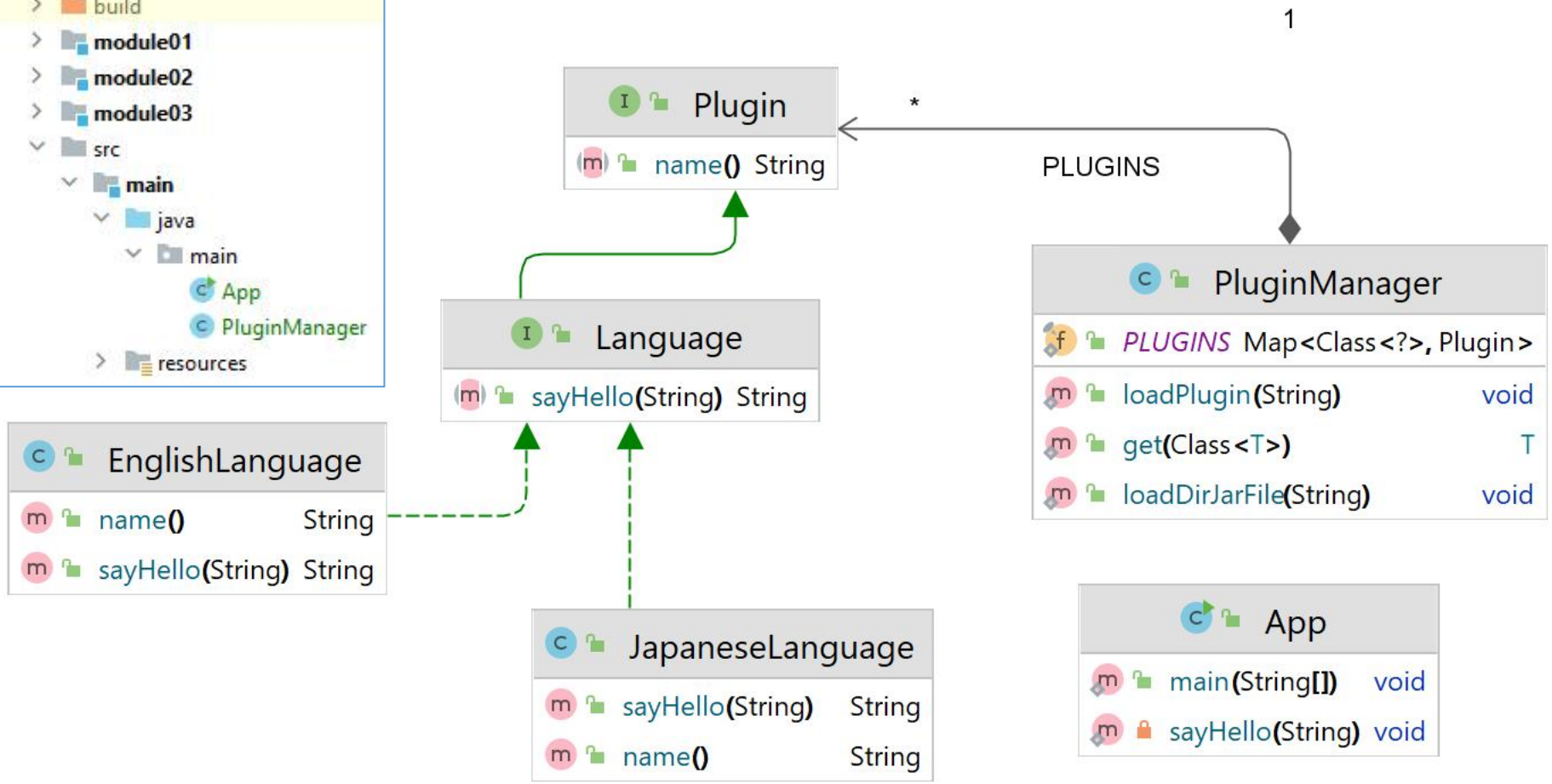
- The core system needs to know about which plug-in modules are available and how to get to them.
  - One common way of implementing this is through a plug-in registry.
  - This registry contains information about each plug-in module, including things like its name, data contract, and remote access protocol details (depending on how the plug-in is connected to the core system).
- The registry can be as simple as an internal map structure owned by the core system containing a key and the plug-in component reference, or it can be as complex as a registry and discovery tool either embedded within the core system or deployed externally (such as Apache ZooKeeper or Consul).

```
Map<String, String> registry = new HashMap<String, String>();
```

Microkernel Architecture Style

Rating

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★★
Reliability	★★★
Scalability	★
Simplicity	★★★★
Testability	★★★

Microkernel Architecture Style

Example